

# Java Fundamentals

## COMP1039 Programming Paradigms

---

Yuan Cheng

Spring 2026

School of Computer Science  
The University of Nottingham Ningbo China

# Java Fundamentals

---

# Program Control Statements

## Controlling a Program's Flow of Execution

Java provides three categories of program control statements:

- **Selection Statements**
  - Choose between different paths of execution
  - Examples: `if`, `if-else`, `switch`
- **Iteration Statements**
  - Repeat a section of code
  - Examples: `for`, `while`, `do-while`
- **Jump Statements**
  - Directly transfer control to another location
  - Examples: `break`, `continue`, `return`

# The if Statement

## Simplest Form

---

```
1 if (condition)
2     statement;
```

---

## Example

---

```
1 if (3 < 4)
2     System.out.println("yes");
```

---

## The if-else Form

---

```
1 if (condition)
2     statement1;
3 else
4     statement2;
```

---

## Example: if Statements

---

```
1  class IfDemo {  
2      public static void main(String[] args) {  
3          int a, b;  
4          a = 2;  
5          b = 3;  
6  
7          if (a < b)  
8              System.out.println("a is less than b");  
9  
10         if (a == b)  
11             System.out.println("you won't see this");  
12     }  
13 }
```

---

**Question: What is the problem?**

- Is there a logical issue?
- What will be printed?

# Nested if Statements

---

```
1  class IfDemo {
2      public static void main(String[] args) {
3
4          int a = 2;
5          int b = 3;
6
7          if (a < b)
8              System.out.println("a is less than b");
9          else {
10             if (a == b)
11                 System.out.println("a equals b");
12             else
13                 System.out.println("a is larger than b");
14         }
15     }
16 }
```

---

# The if-else-if Ladder

## General Form

---

```
1  if (condition1)
2      statement1;
3  else if (condition2)
4      statement2;
5  else if (condition3)
6      statement3;
7  ...
8  else
9      statementN;
```

---

## Key Points

- Conditions are checked **top to bottom**
- Only the **first true condition** executes
- The `else` block runs if all conditions are false

# Revised Example: if-else-if Ladder

```
1  class IfDemo {
2      public static void main(String[] args) {
3
4          int a = 2;
5          int b = 3;
6
7          if (a < b)
8              System.out.println("a is less than b");
9          else if (a == b)
10             System.out.println("a equals b");
11         else
12             System.out.println("a is larger than b");
13     }
14 }
```

## Why is this better?

- Cleaner than nested if statements
- Easier to read and maintain
- Only one branch executes

## Example: Multiple else-if Conditions

---

```
1  int season = 0;
2  String seasonString;
3
4  if (season == 1) {
5      seasonString = "Spring";
6  } else if (season == 2) {
7      seasonString = "Summer";
8  } else if (season == 3) {
9      seasonString = "Autumn";
10 } else if (season == 4) {
11     seasonString = "Winter";
12 } else {
13     seasonString = "Not a season";
14 }
15
16 System.out.println(seasonString);
```

---

### Observation:

- Many equality checks on the same variable
- Code works, but becomes long and repetitive

# The switch Statement

## General Form

---

```
1  switch (expression) {  
2      case constant1:  
3          statement sequence;  
4          break;  
5      case constant2:  
6          statement sequence;  
7          break;  
8      ...  
9      default:  
10         statement sequence;  
11         break;  
12 }
```

---

## Notes

- Expression type: byte, short, int, char, enum, or String
- break prevents fall-through
- default executes if no case matches

# Modern switch (Java 14+)

---

```
1 public class Season {
2     public static void main(String[] args) {
3
4         int season = 0;
5
6         String seasonString = switch (season) {
7             case 1 -> "Spring";
8             case 2 -> "Summer";
9             case 3 -> "Autumn (Winter is Coming)";
10            case 4 -> "Winter";
11            default -> "Not a season";
12        };
13
14        System.out.println(seasonString);
15    }
16 }
```

---

Why is this better?

## Exercise 1: switch Fall-Through

**Question:** Given the following code, what will be the final value of `y`?

### Code

---

```
1  int x = 2;
2  int y = 0;
3
4  switch(x) {
5      case 1: y += 1; break;
6      case 2: y += 2;
7      case 3: y += 3;
8  }
```

---

**Think carefully:**

- Does every case stop automatically?
- What happens when there is no `break`?

## Iteration Statements

Loops allow a block of code to be executed repeatedly.

## Basic Loops

- **for** loop
- **while** loop
- **do-while** loop

## More Advanced for Loops

- Labelled for
- Enhanced for (for-each loop)

# The for Loop

## General Form

---

```
1  for (initialization; condition; iteration)
2      statement;
3
4  for (initialization; condition; iteration) {
5      statement sequence;
6  }
```

---

## Loop Components

- **Initialization** — sets the loop control variable
- **Condition** — determines when the loop stops
- **Iteration** — updates the loop control variable

## Example: for Loop

```
1  class ForDemo {  
2      public static void main(String[] args) {  
3  
4          int count;  
5  
6          for (count = 0; count < 5; count = count + 1)  
7              System.out.println("This is count: " + count);  
8  
9          System.out.println("Done!");  
10     }  
11 }
```

### What happens?

- Loop runs while `count < 5`
- Prints values: 0, 1, 2, 3, 4
- After loop ends, prints "Done!"

# Variations of the for Loop

## Standard Form

---

```
1  for (initialization; condition; iteration) {  
2      statement sequence;  
3  }
```

---

## Initialization Outside the Loop

---

```
1  initialization;  
2  
3  for (; condition; iteration) {  
4      statement sequence;  
5  }
```

---

## Other Variations

- Any of the three parts can be omitted

# for Loop with All Three Parts Omitted

---

```
1 for (;;) {  
2     // statements  
3 }
```

---

- All three components are omitted:
  - Initialization
  - Condition
  - Iteration
- No condition → defaults to true
- Result: **Infinite loop**
- Equivalent to:

---

```
1 while (true) {  
2     // statements  
3 }
```

---

# The while Loop

## General Form

---

```
1 while (condition)
2     statement;
```

---

## Block Form

---

```
1 while (condition) {
2     statement sequence;
3 }
```

---

- The condition is evaluated **before** each iteration
- If the condition is **false**, the loop stops
- Called a **pre-test loop**

## Example: while Loop

---

```
1  class WhileDemo {
2      public static void main(String[] args) {
3          int count = 0;
4
5          while (count < 5) {
6              System.out.println("Count is: " + count);
7              count++;
8          }
9      }
10 }
```

---

- Initialization: `int count = 0;`
- Condition: `count < 5`
- Update: `count++`

# The Do-While Loop

## General Form

---

```
1 do {  
2     statement sequence;  
3 } while (condition);
```

---

## Example

---

```
1 class DoWhileDemo {  
2     public static void main(String[] args) {  
3         int count = 0;  
4  
5         do {  
6             System.out.println("Count is: " + count);  
7             count++;  
8         } while (count < 5);  
9     }  
10 }
```

- Take **1–2 minutes** to reflect on this lecture.
- On a sheet of paper, write:
  - **One thing you learned** in this lecture
  - **One thing you didn't understand**